Practice 1: Implement Image Transformation in OpenCV

Aim

To perform resizing, rotation, translation and flipping of an image using OpenCV.

Procedure

1. Create a sample image.

2. Apply resize, rotation and translation matrices.

3. Flip the image horizontally.

4. Display the original and transformed images.

Program

```
import cv2
import numpy as np

img = np.zeros((400, 600, 3), np.uint8)

cv2.rectangle(img, (100, 100), (300, 250), (255, 255, 255), -1)

h, w = img.shape[:2]

rot = cv2.warpAffine(
    img,
    cv2.getRotationMatrix2D((w // 2, h // 2), 30, 1),
    (w, h)
)

trans = cv2.warpAffine(
    img,
    np.float32([[1, 0, 50], [0, 1, 40]]),
    (w, h)
)

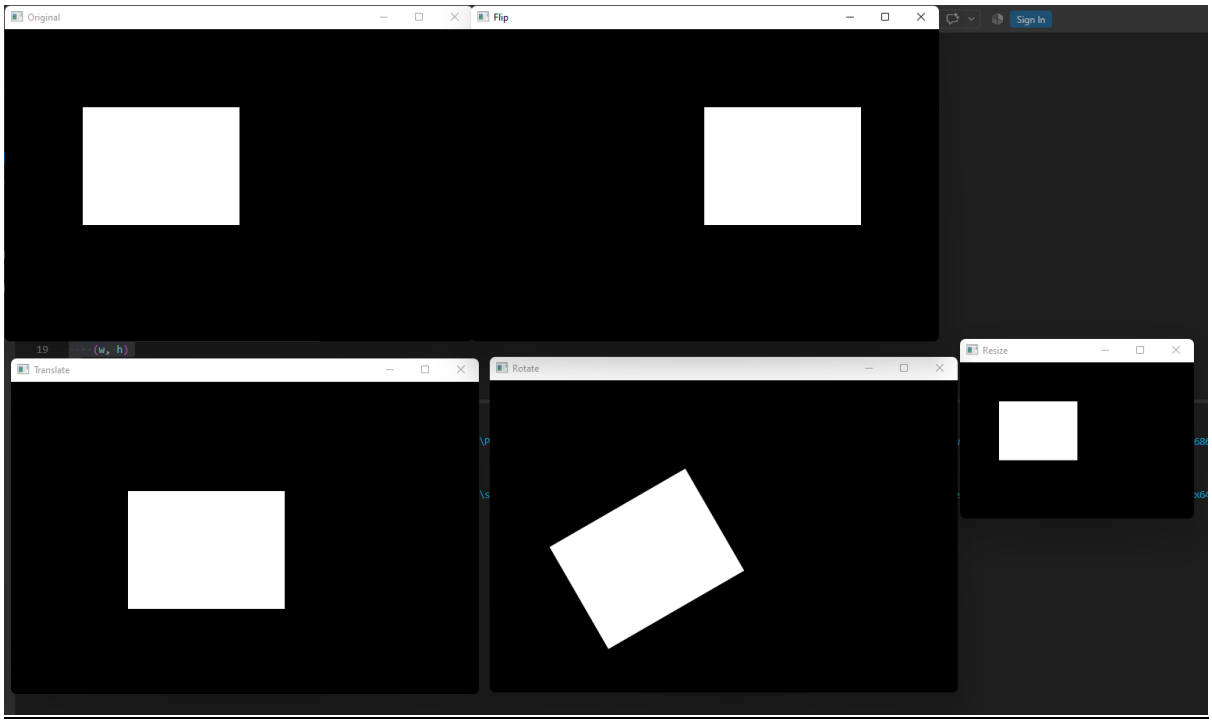
resize = cv2.resize(img, (300, 200))

flip = cv2.flip(img, 1)

for n, x in [
    ("Original", img),
    ("Resize", resize),
    ("Rotate", rot),
    ("Translate", trans),
    ("Flip", flip)
]:
    cv2.imshow(n, x)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Image transformations were successfully implemented using OpenCV.

Practice 2: Implement Projection

Aim

To implement a 2-D perspective/projective transformation.

Procedure

1. Create an image with a rectangular object.

2. Specify four source and destination points.

3. Calculate the perspective matrix.

4. Apply warpPerspective() and display the result.

Program

```
import cv2
import numpy as np

img = np.zeros((400, 600, 3), np.uint8)

cv2.rectangle(img, (100, 80), (500, 320), (255, 255, 255), 3)

src = np.float32([
    [100, 80],
    [500, 80],
    [500, 320],
    [100, 320]
])

dst = np.float32([
    [60, 100],
    [540, 60],
    [500, 340],
    [100, 300]
])

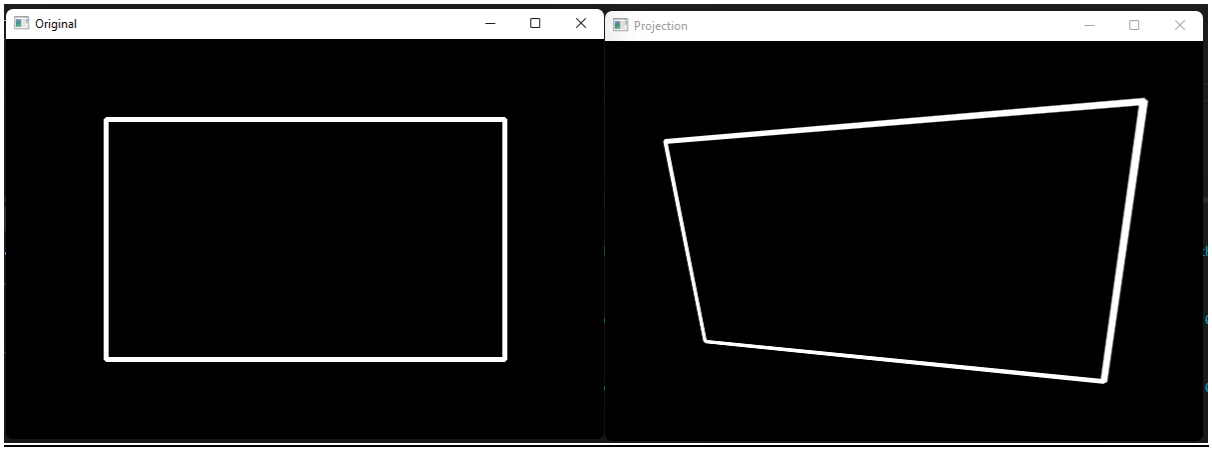
M = cv2.getPerspectiveTransform(src, dst)

out = cv2.warpPerspective(img, M, (600, 400))

cv2.imshow("Original", img)
cv2.imshow("Projection", out)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Perspective projection was successfully implemented.

Practice 3: Implementation of Quantization

Aim

To reduce the number of intensity levels in an image.

Procedure

1. Create a grayscale gradient.
2. Select four intensity levels.
3. Map pixels to the nearest quantization level.
4. Display both images.

Program:

```
import cv2
import numpy as np

img = np.tile(np.arange(256, dtype=np.uint8), (256, 1))

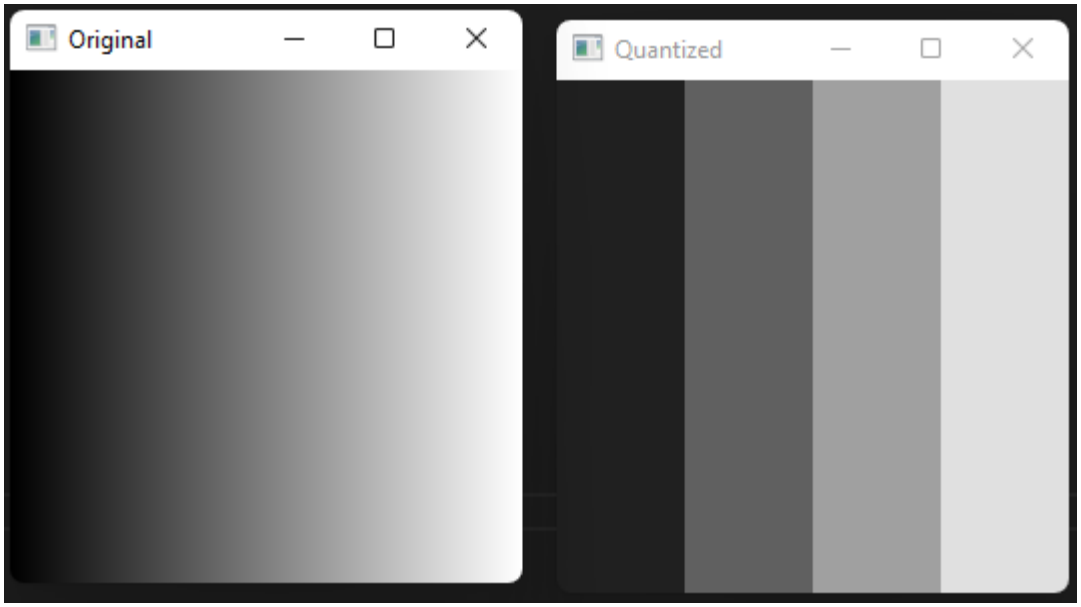
levels = 4
step = 256 // levels

q = np.clip(
    (img // step) * step + step // 2,
    0,
    255
).astype(np.uint8)

cv2.imshow("Original", img)
cv2.imshow("Quantized", q)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:



Result:

Image quantization was successfully performed.

Practice 4: Edge Detection and Feature Extraction

Aim

To detect edges and extract corner features using Canny, LoG-style filtering and Harris detection.

Procedure

1. Create a geometric image.

2. Convert it to grayscale and blur it.

3. Apply Canny and Laplacian edge detection.

4. Apply Harris corner detection.

5. Display the results.

Program

```
import cv2
import numpy as np

img = np.zeros((400, 600, 3), np.uint8)

# Draw shapes
cv2.rectangle(img, (80, 80), (300, 300), (255, 255, 255), -1)
cv2.circle(img, (470, 190), 90, (180, 180, 180), -1)

# Grayscale and Gaussian blur
g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
b = cv2.GaussianBlur(g, (5, 5), 0)

# Canny edge detection
canny = cv2.Canny(b, 50, 150)

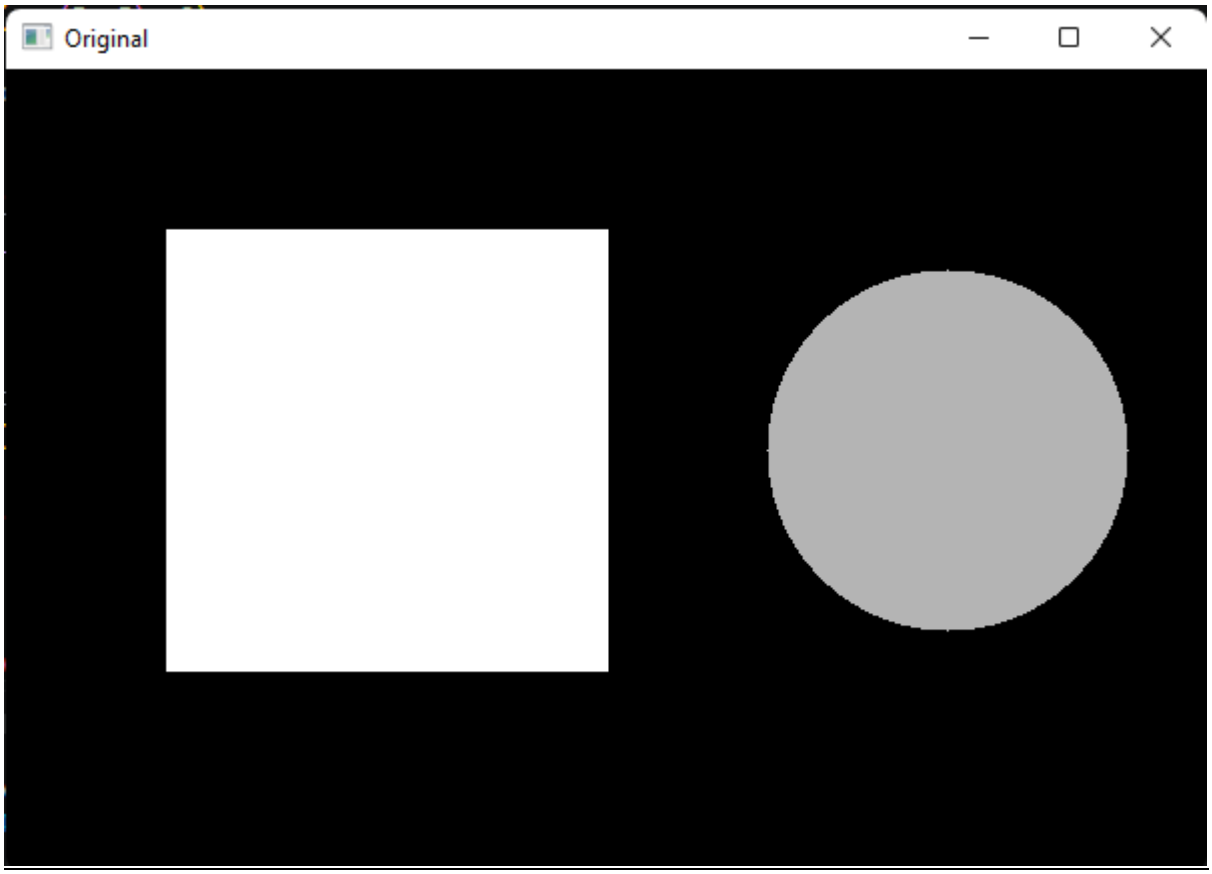
# Laplacian of Gaussian (LoG)
log = cv2.convertScaleAbs(
    cv2.Laplacian(b, cv2.CV_64F)
)

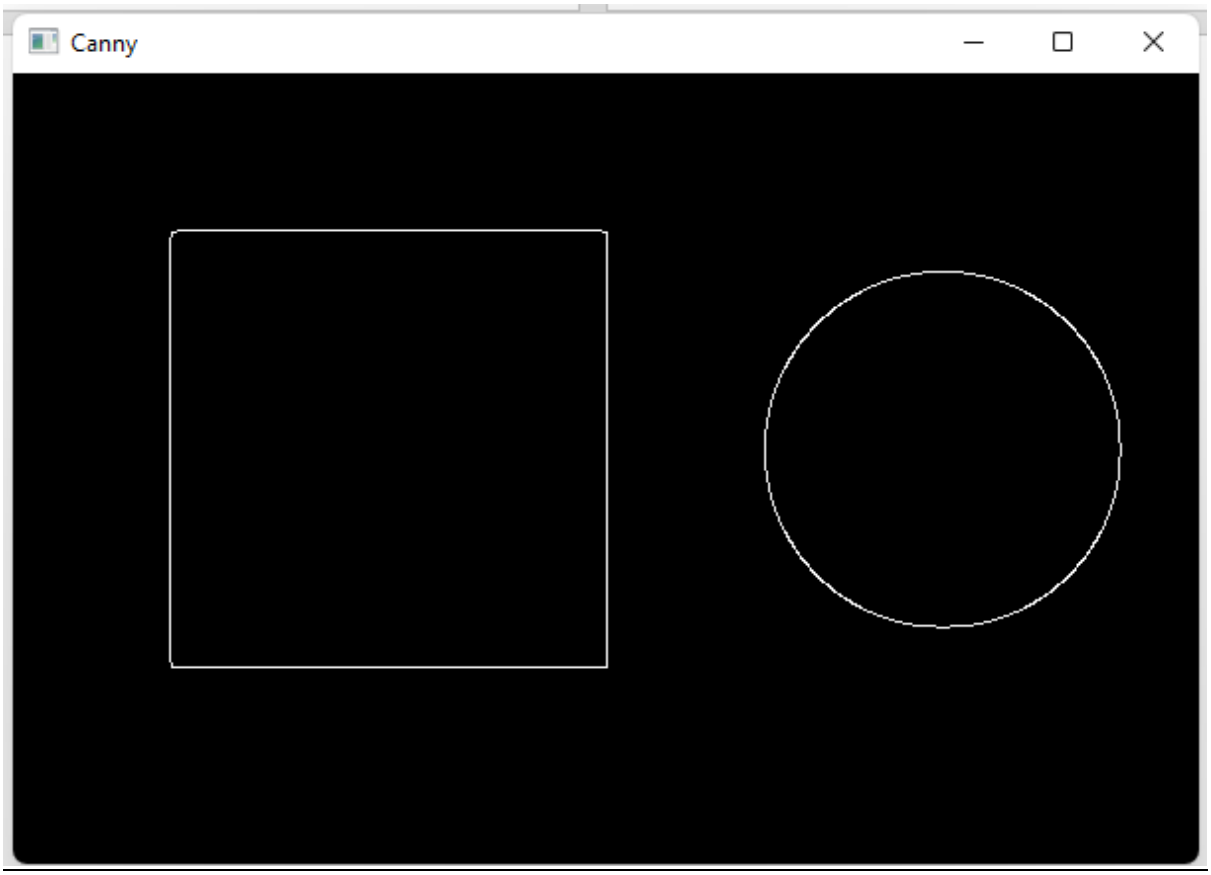
# Harris corner detection
h = cv2.cornerHarris(np.float32(g), 2, 3, 0.04)
corners = img.copy()
corners[h > 0.01 * h.max()] = (0, 0, 255)

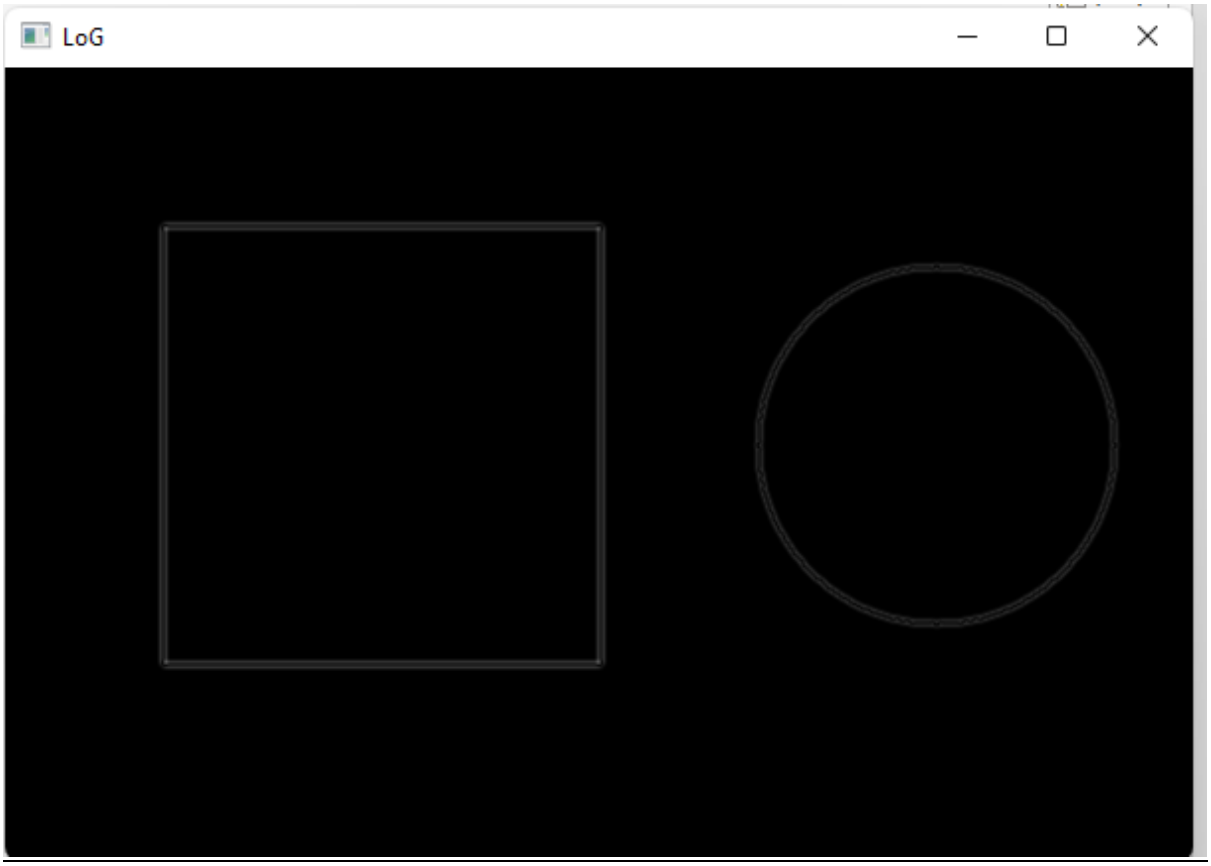
# Display results
```

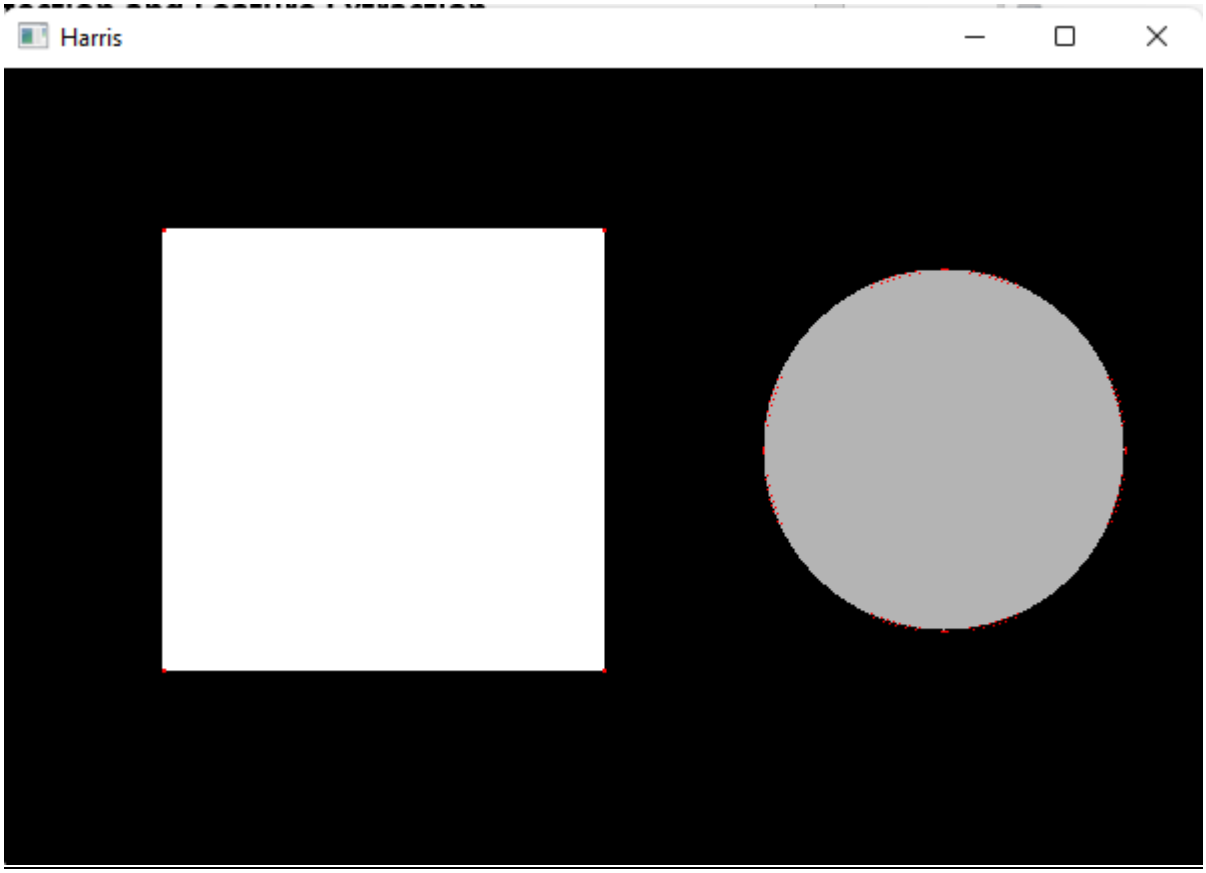
```
for n, x in [
    ("Original", img),
    ("Canny", canny),
    ("LoG", log),
    ("Harris", corners)
]:
    cv2.imshow(n, x)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Output




Canny


LoG



Result

Edge detection and corner detection were successfully performed using Canny, LoG, and Harris corner detection techniques.

Practice 5: Implementation of SIFT and SURF

Aim

To detect local features using SIFT and safely test SURF availability.

Procedure

1. Create a sample image.

2. Detect SIFT keypoints and descriptors.

3. Attempt SURF only when supported by the installed OpenCV build.

4. Display detected keypoints.

Program

```
import cv2
import numpy as np

img = np.zeros((400, 600, 3), np.uint8)

cv2.rectangle(img, (80, 80), (300, 300), (255, 255, 255), 3)
cv2.circle(img, (450, 180), 80, (180, 180, 180), 3)

g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# SIFT
s = cv2.SIFT_create()
kp, des = s.detectAndCompute(g, None)

sift = cv2.drawKeypoints(
    img, kp, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

print("SIFT keypoints:", len(kp))

# SURF
surf = img.copy()

try:
    if hasattr(cv2, "xfeatures2d") and hasattr(cv2.xfeatures2d,
"SURF_create"):
        z = cv2.xfeatures2d.SURF_create(400)
        sk, sd = z.detectAndCompute(g, None)

        surf = cv2.drawKeypoints(img, sk, None)
```

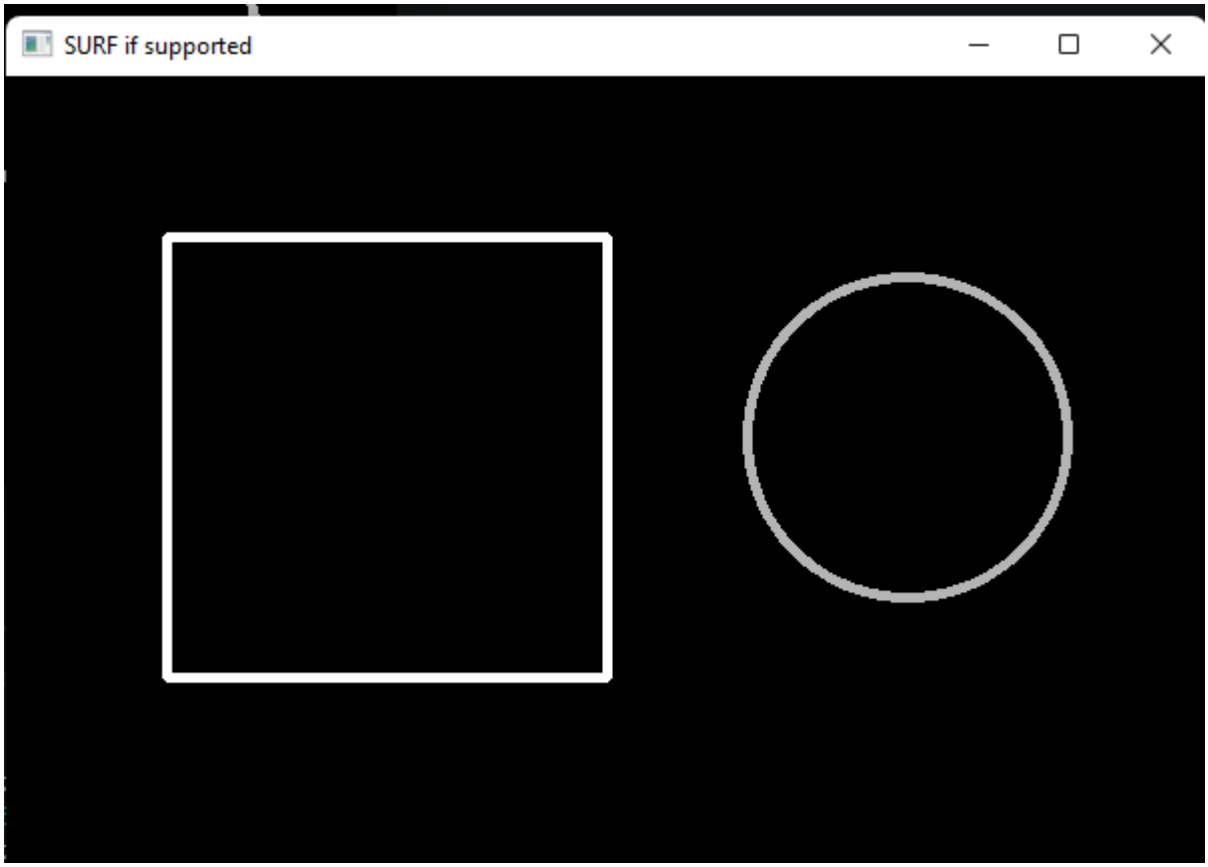
```
        print("SURF keypoints:", len(sk))
    else:
        print("SURF is not available in this OpenCV build.")

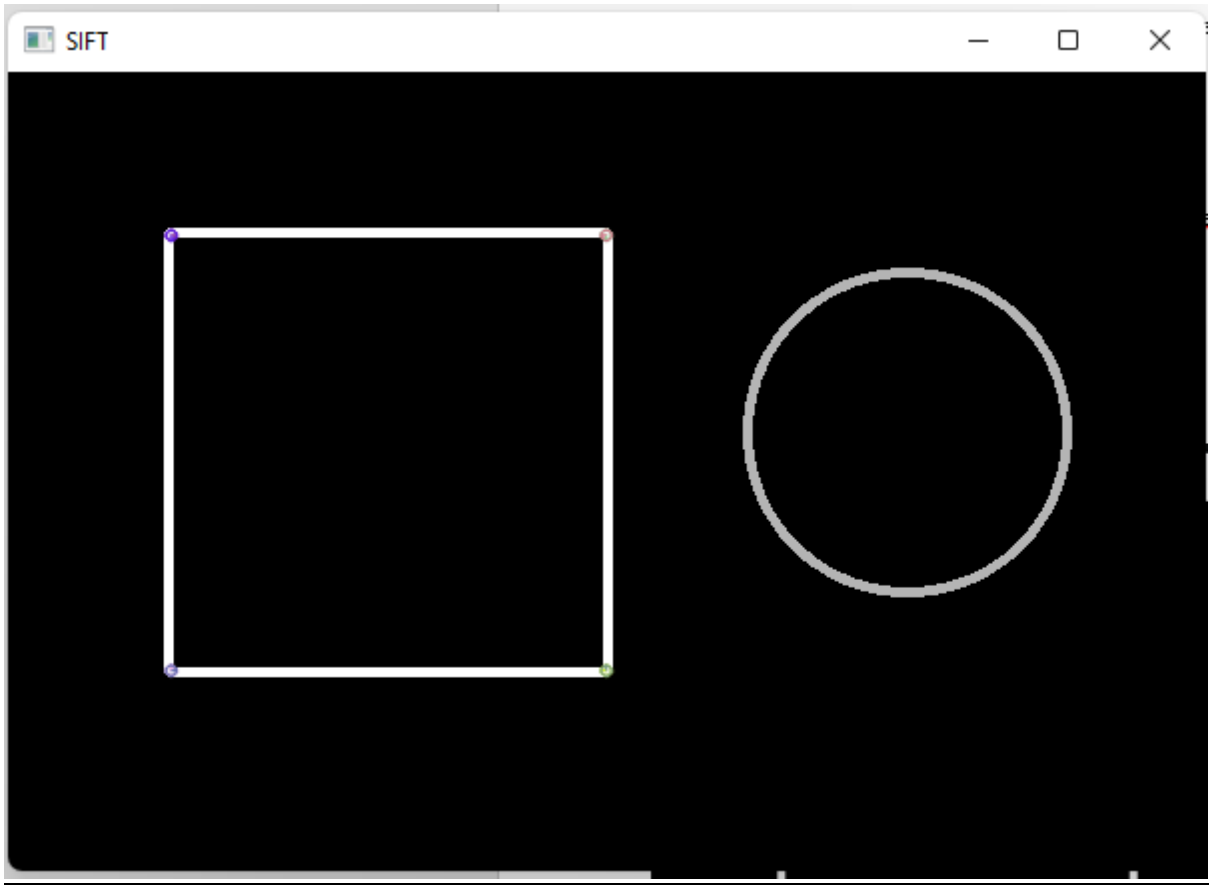
except cv2.error:
    print("SURF is not available; SIFT completed.")

cv2.imshow("SIFT", sift)
cv2.imshow("SURF if supported", surf)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output





Result

SIFT feature extraction was successfully implemented with safe SURF availability checking.

Practice 6: Implement Image Segmentation

Aim

To segment an image using thresholding, region growing and mean-shift processing.

Procedure

1. Create an image containing objects.

2. Apply binary thresholding.

3. Use flood fill as a simple region-growing method.

4. Apply mean-shift filtering.

5. Display the results.

Program

```
import cv2
import numpy as np

img = np.zeros((400, 600, 3), np.uint8)

cv2.circle(img, (180, 200), 90, (255, 255, 255), -1)
cv2.rectangle(img, (330, 100), (520, 300), (170, 170, 170), -1)

g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Thresholding
_, th = cv2.threshold(g, 100, 255, cv2.THRESH_BINARY)

# Region Growing
mask = np.zeros((g.shape[0] + 2, g.shape[1] + 2), np.uint8)
region = g.copy()

cv2.floodFill(
    region, mask, (180, 200), 255,
    loDiff=10, upDiff=10
)

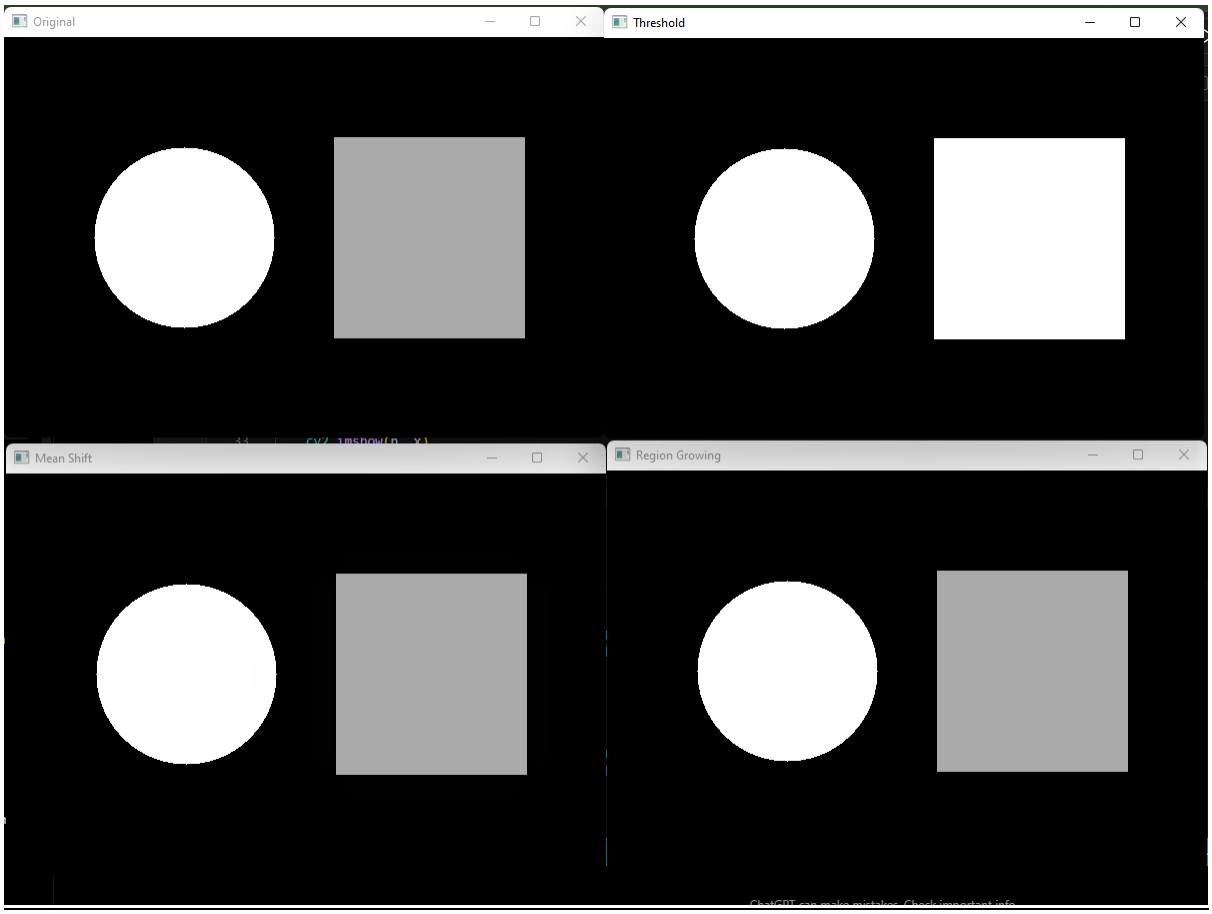
# Mean Shift
ms = cv2.pyrMeanShiftFiltering(img, 21, 51)

# Display results
for n, x in [
    ("Original", img),
    ("Threshold", th),
    ("Region Growing", region),
```

```
    ("Mean Shift", ms)
]:
    cv2.imshow(n, x)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Resul;t

Basic image segmentation techniques were successfully implemented.

Practice 7: Implement Object Detection

Aim

To detect objects using contours and bounding rectangles.

Procedure

1. Create multiple objects.

2. Convert the image to binary form.

3. Find external contours.

4. Ignore very small contours.

5. Draw bounding boxes and labels.

Program

```
import cv2
import numpy as np

img = np.zeros((450, 700, 3), np.uint8)

cv2.rectangle(img, (70, 80), (250, 280), (255, 255, 255), -1)
cv2.circle(img, (430, 180), 85, (200, 200, 200), -1)
cv2.rectangle(img, (500, 300), (650, 400), (150, 150, 150), -1)

g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, b = cv2.threshold(g, 50, 255, cv2.THRESH_BINARY)

cs, _ = cv2.findContours(
    b,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

out = img.copy()
count = 0

for c in cs:
    if cv2.contourArea(c) > 1000:
        x, y, w, h = cv2.boundingRect(c)
        count += 1

        cv2.rectangle(
            out,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
```

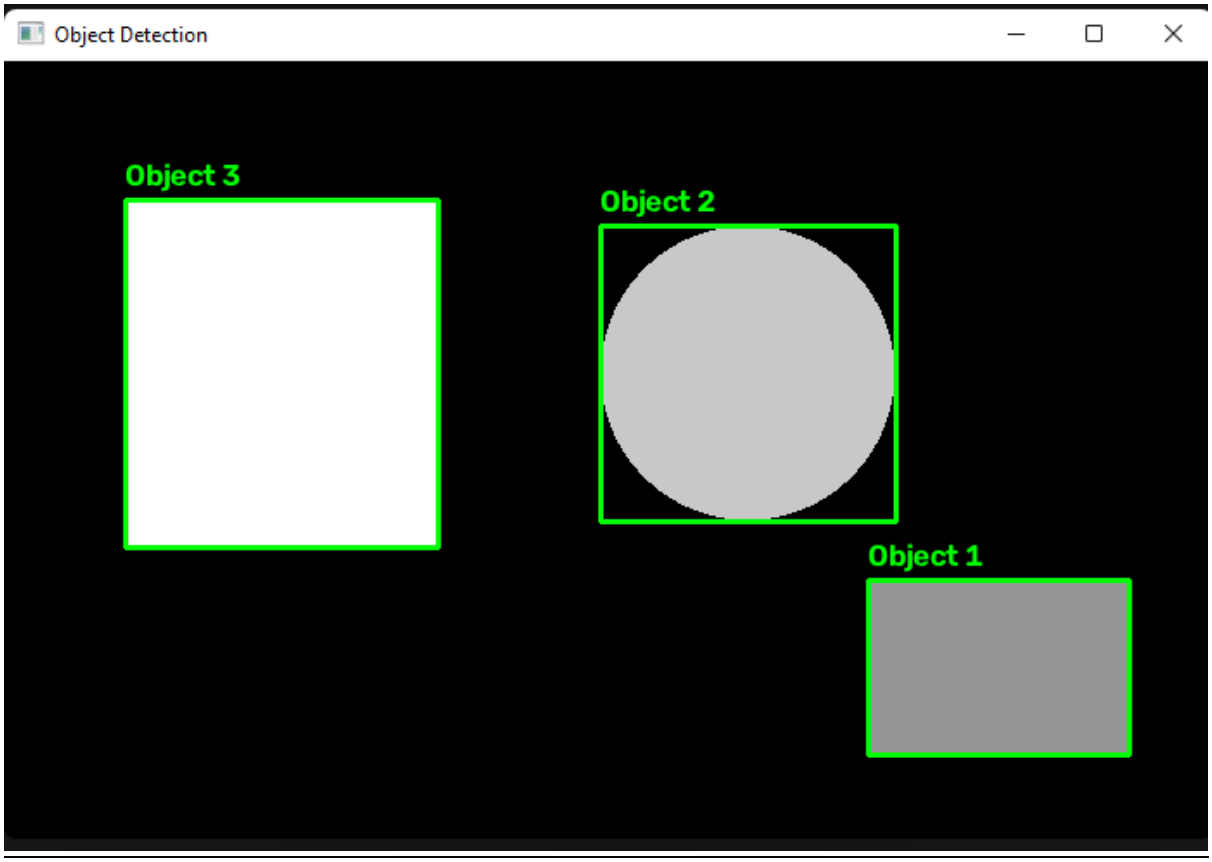
```
            2
        )

        cv2.putText(
            out,
            f"Object {count}",
            (x, y - 8),
            0,
            0.6,
            (0, 255, 0),
            2
        )

print("Objects detected:", count)

cv2.imshow("Object Detection", out)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output



Result

Contour-based object detection was successfully implemented.